VPS Online Hosting — Online Master on Ubuntu (NGINX + HTTPS + domain)

Step-by-step guide to deploy the **online master** (central server: Django API + PostgreSQL + Angular frontend) on a single **Ubuntu VPS**, served over **HTTPS** with a **domain name** (not an IP), using **NGINX** + **Let's Encrypt (certbot)**.

Architecture used by this guide - **Docker** runs only: `db` (PostgreSQL) + `web` (Django/Gunicorn, bound to `127.0.0.1:8000`). - **NGINX on the host** terminates TLS, serves the Angular build + uploaded media, and reverse-proxies `/api`, `/admin`, `/static` to the Django container. - **certbot (nginx plugin)** issues the certificate, adds the HTTPS server block, the http→https redirect, and sets up **automatic renewal**.

Throughout, replace `erp.mondomaine.com` with your real domain and `admin@mondomaine.com` with your email.

Two modes: pick one

- **Mode B — Domain + HTTPS** (recommended, the main steps below). You own a domain and get a real certificate from Let's Encrypt.
- **Mode A — IP only, HTTP** (no domain yet). You reach the app at `http://YOUR_VPS_IP`. **No HTTPS**: Let's Encrypt does **not** issue certificates for a bare IP. Use this to get running now, then switch to Mode B when you buy a domain.

Mode A — IP only (HTTP, no domain yet)

Follow all the steps below **with these changes**, and **skip steps 1 and 10** (DNS + certbot):

1. **Step 5 (frontend API URL)** — set it to a relative path so it works on any IP: `ts apiUrl: '/api/v1',`
2. **Step 7 (`.env`)** — use your IP and **disable the HTTPS enforcement** (otherwise Django force-redirects to https and login breaks): `dotenv ALLOWED_HOSTS=203.0.113.10 CSRF_TRUSTED_ORIGINS=http://203.0.113.10 SECURE_SSL_REDIRECT=0 SESSION_COOKIE_SECURE=0 CSRF_COOKIE_SECURE=0` (Replace `203.0.113.10` with your real VPS IP.)
3. **Step 9 (NGINX)** — install the **IP** vhost instead of the domain one: `bash sudo cp /opt/erp/deploy/vps/nginx-erp-ip.conf /etc/nginx/sites-available/erp` (No domain to edit — `server_name _;` already catches any IP. Fix the `/opt/erp` paths only if you cloned elsewhere.)
4. **Skip step 10** (certbot). You're done after step 9 — open `http://YOUR_VPS_IP`.

Later, to upgrade to a domain + HTTPS: buy the domain, do step 1 (DNS A record), set `apiUrl` back to `https://your.domain/api/v1` and rebuild (steps 5–6), put the domain back in `.env` and **remove** the three `SECURE_*=0` / cookie lines (step 7), swap the vhost to `nginx-erp.conf` with your domain (step 9), then run certbot (step 10).

Want HTTPS without a domain? (securing an IP deploy)

"No domain" doesn't have to mean "no HTTPS". Two options:

Option 1 — Trusted HTTPS via a free `sslip.io` hostname (recommended)

`sslip.io` (and `nip.io`) resolve `203-0-113-10.sslip.io` → `203.0.113.10` automatically, with no DNS setup. Because it's a real hostname, **Let's Encrypt issues a normal trusted certificate** for it. You get a real padlock + auto-renew, for free.

Just do the **Mode B** steps (the main guide), using `YOUR-IP-with-dashes.sslip.io` as the "domain" everywhere: - **Step 5:** `apiUrl: 'https://203-0-113-10.sslip.io/api/v1'` - **Step 7** `.env`**:** `ALLOWED_HOSTS=203-0-113-10.sslip.io` / `CSRF_TRUSTED_ORIGINS=https://203-0-113-10.sslip.io` (keep the `SECURE_*` at their secure defaults — do **not** set them to 0). - **Step 9:** use `nginx-erp.conf`, set `server_name 203-0-113-10.sslip.io;` - **Step 10:** `sudo certbot --nginx -d 203-0-113-10.sslip.io -m you@email --agree-tos --redirect`

That's genuine, browser-trusted HTTPS pointing at your IP, no purchase.

Option 2 — Self-signed certificate on the raw IP (`https://YOUR_IP`)

Real encryption, URL stays the bare IP, but **browsers show a "Not secure / proceed anyway" warning** (the cert isn't signed by a trusted authority). OK for internal/admin use.

Follow Mode A, but at **step 9** do this instead:

```
# 1) Generate a self-signed cert for your IP (replace 203.0.113.10)
sudo mkdir -p /etc/ssl/erp
sudo openssl req -x509 -nodes -days 825 -newkey rsa:2048 \
  -keyout /etc/ssl/erp/selfsigned.key -out /etc/ssl/erp/selfsigned.crt \
  -subj "/CN=203.0.113.10" -addext "subjectAltName=IP:203.0.113.10"

# 2) Install the HTTPS (self-signed) vhost
sudo cp /opt/erp/deploy/vps/nginx-erp-ip-https.conf /etc/nginx/sites-available/erp
sudo ln -sf /etc/nginx/sites-available/erp /etc/nginx/sites-enabled/erp
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

In **step 7** `.env` for Option 2, the connection *is* HTTPS, so keep secure cookies but turn **off HSTS** (never pin a self-signed cert) and let NGINX do the redirect:

```
ALLOWED_HOSTS=203.0.113.10
CSRF_TRUSTED_ORIGINS=https://203.0.113.10
SECURE_SSL_REDIRECT=0          # NGINX already redirects 80 -> 443
SECURE_HSTS_SECONDS=0          # do NOT HSTS-pin a self-signed cert
# leave SESSION_COOKIE_SECURE / CSRF_COOKIE_SECURE at their secure defaults (1)
```

Open `https://203.0.113.10` and accept the browser warning once.

> ⚠️ **If `docker compose ... up --build` fails on `apt-get` with `Temporary failure resolving 'deb.debian.org'`** — that's a Docker **DNS** problem on the VPS, not a config issue. Fix it before anything else: see **Troubleshooting → Docker build can't resolve apt mirrors**.

---

0. Prerequisites

- An **Ubuntu 22.04 / 24.04** VPS with root or sudo access, **≥ 2 GB RAM** recommended (building the Angular frontend is memory-hungry — see step 6 if you have less).

- A **domain name** you control (e.g. `mondomaine.com`) and access to its DNS.
- The ability to pull this repository on the VPS (it is private — use a **GitHub deploy key** or a **Personal Access Token**; alternatively build/clone locally and `scp`).

---

1. Point your domain at the VPS (DNS)

In your domain registrar / DNS provider, create an **A record**:

Type	Name (host)	Value (points to)
A	`erp` (or `@`)	**your VPS public IP**

- `erp.mondomaine.com` → `203.0.113.10` (your VPS IP).
- If you also want `www`, add an A record for `www` too (optional).

Wait for DNS to propagate, then verify from your laptop:

```
dig +short erp.mondomaine.com      # must return your VPS IP
```

Do **not** continue to the HTTPS step until this returns the correct IP (Let's Encrypt validates the domain over the public internet).

---

2. Prepare the VPS (packages, firewall)

SSH into the VPS, then:

```
sudo apt update && sudo apt upgrade -y

# Firewall: allow SSH, HTTP, HTTPS
sudo apt install -y ufw
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable
sudo ufw status
```

> If your provider has its own cloud firewall/security group, open **22, 80, 443** there too.

---

3. Install Docker + Docker Compose

```
# Docker Engine + Compose plugin (official convenience script)
curl -fsSL https://get.docker.com | sudo sh

# Run docker without sudo (log out/in afterwards, or run `newgrp docker`)
sudo usermod -aG docker $USER
newgrp docker

docker --version
docker compose version
```

---

4. Get the code

Clone the repository to `/opt/erp` (the paths in the nginx config assume this):

```
sudo mkdir -p /opt/erp
sudo chown $USER:$USER /opt/erp

# Private repo — use your deploy key or a PAT in the URL:
git clone https://github.com/zealot19/brainstore.git /opt/erp
cd /opt/erp
```

After cloning you should have `/opt/erp/backend`, `/opt/erp/angular`, `/opt/erp/deploy/vps`.

> If you cloned somewhere else, edit the two `root` / `alias` paths in `deploy/vps/nginx-erp.conf` accordingly.

---

5. Point the frontend at your domain

The production Angular build bakes in the API URL. Edit it to your HTTPS domain:

```
nano /opt/erp/angular/src/environments/environment.prod.ts
```

Set:

```
  apiUrl: 'https://erp.mondomaine.com/api/v1',
```

(Front and API are on the same domain. You could also use the relative value `'/api/v1'` to avoid hard-coding the domain.)

---

6. Build the Angular frontend

Build using a throwaway Node container (no need to install Node on the host). Output goes to `angular/dist/angular/browser`, which NGINX will serve.

```
cd /opt/erp
docker run --rm \
  -e NODE_OPTIONS=--max_old_space_size=4096 \
  -v "$PWD/angular:/app" -w /app node:20-alpine \
  sh -c "npm ci && npx ng build --configuration production"

ls angular/dist/angular/browser/index.html   # must exist
```

> **Low RAM (< 2 GB)?** The build may be killed (OOM). Either add swap first: `bash sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile sudo mkswap /swapfile && sudo swapon /swapfile echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab` …or build on your laptop and copy `angular/dist` to the VPS with `scp -r`.

---

7. Configure the backend environment (`.env`)

```
cd /opt/erp/deploy/vps
cp .env.example .env
nano .env
```

Fill it in. Minimum required for a working HTTPS deploy:

```
SECRET_KEY=<paste a long random string>           # see command below
DJANGO_SETTINGS_MODULE=config.settings.production
DEBUG=0
ALLOWED_HOSTS=erp.mondomaine.com
CSRF_TRUSTED_ORIGINS=https://erp.mondomaine.com

POSTGRES_DB=erp
POSTGRES_USER=erp
POSTGRES_PASSWORD=<a strong password>
DATABASE_URL=postgres://erp:<the same strong password>@db:5432/erp

LANGUAGE_CODE=fr-fr
TIME_ZONE=Africa/Douala

# Central server (keep if agency nodes will sync to this master)
SYNC_ENABLED=1
NODE_ROLE=master
SYNC_TOKEN=<a strong shared token>

# Online payments simulated until Campay is configured
FAKE_PAYMENTS=1
```

Generate a strong `SECRET_KEY`:

```
python3 -c "import secrets; print(secrets.token_urlsafe(50))"
```

> The database password appears **twice** (in `POSTGRES_PASSWORD` and inside `DATABASE_URL`) — they must match. `ALLOWED_HOSTS` / `CSRF_TRUSTED_ORIGINS` must be your real domain. The production

> settings already trust the `X-Forwarded-Proto` header from NGINX, so HTTPS detection works behind the proxy.

---

8. Start the database + backend

```
cd /opt/erp/deploy/vps
docker compose -f docker-compose.nginx.yml up -d --build
```

- The backend image builds, then on startup it runs `migrate` automatically and starts Gunicorn on `127.0.0.1:8000`.
- Check it's healthy:

```
docker compose -f docker-compose.nginx.yml ps
docker compose -f docker-compose.nginx.yml logs -f web   # Ctrl+C to stop following
curl -I http://127.0.0.1:8000/api/           # from the VPS: should respond (e.g.
200/401/404, not "connection refused")
```

Create the first admin user:

```
docker compose -f docker-compose.nginx.yml exec web python manage.py createsuperuser
```

---

9. Install and configure NGINX (host)

```
sudo apt install -y nginx

# Install the site config shipped in the repo
sudo cp /opt/erp/deploy/vps/nginx-erp.conf /etc/nginx/sites-available/erp

# Edit it: set your domain (and fix the /opt/erp paths if you cloned elsewhere)
sudo nano /etc/nginx/sites-available/erp        # replace erp.mondomaine.com

# Enable it, disable the default site
sudo ln -sf /etc/nginx/sites-available/erp /etc/nginx/sites-enabled/erp
sudo rm -f /etc/nginx/sites-enabled/default

# NGINX (www-data) must be able to read the Angular build + media
sudo chmod o+x /opt /opt/erp /opt/erp/angular /opt/erp/deploy /opt/erp/deploy/vps

# Test config and reload
sudo nginx -t
sudo systemctl reload nginx
```

Now visit **`http://erp.mondomaine.com`** in a browser — the app should load over plain HTTP. (If it doesn't, see Troubleshooting before enabling HTTPS.)

---

10. Enable HTTPS with Let's Encrypt (certbot)

```
sudo apt install -y certbot python3-certbot-nginx

sudo certbot --nginx -d erp.mondomaine.com -m admin@mondomaine.com --agree-tos --redirect
```

certbot will: - obtain a free certificate for your domain, - rewrite the NGINX config to add the **HTTPS (443)** server block, - add an automatic **http → https redirect** (`--redirect`), - install a **systemd timer** that auto-renews the certificate.

Verify auto-renewal is scheduled and works (dry run):

```
systemctl list-timers | grep certbot
sudo certbot renew --dry-run
```

---

11. Verify

- Open `https://erp.mondomaine.com` → the app loads with a valid padlock.
- `https://erp.mondomaine.com/admin/` → Django admin login (your superuser).
- `http://erp.mondomaine.com` → automatically redirects to `https://` .
- Log in and confirm the UI talks to the API (no CORS/mixed-content errors in the browser console).

You're live. 🎉

---

12. Updating the deployment (new version)

```
cd /opt/erp
git pull

# Rebuild the frontend (only if angular/ changed)
docker run --rm -e NODE_OPTIONS=--max_old_space_size=4096 \
  -v "$PWD/angular:/app" -w /app node:20-alpine \
  sh -c "npm ci && npx ng build --configuration production"

# Rebuild + restart the backend (migrations run automatically on startup)
cd /opt/erp/deploy/vps
docker compose -f docker-compose.nginx.yml up -d --build

# Reload NGINX only if you changed nginx-erp.conf
sudo nginx -t && sudo systemctl reload nginx
```

> The frontend is static files served by NGINX, so a new Angular build is picked up immediately after it finishes (no NGINX reload needed).

---

13. Backups (do this!)

```
# Database dump
cd /opt/erp/deploy/vps
docker compose -f docker-compose.nginx.yml exec -T db \
  pg_dump -U erp erp | gzip > ~/erp-db-$(date +%F).sql.gz

# Uploaded media
tar czf ~/erp-media-$(date +%F).tar.gz -C /opt/erp/deploy/vps media
```

Copy these off the VPS regularly (e.g. `scp` to your machine or an object store).

14. Troubleshooting

Symptom	Check
`http://domain` shows NGINX default / 404	`sudo rm /etc/nginx/sites-enabled/default`, re-enable `erp`, `sudo nginx -t && sudo systemctl reload nginx`.
502 Bad Gateway	Backend not up: `docker compose -f docker-compose.nginx.yml ps` / `logs web`. Is it listening on `127.0.0.1:8000`?
403 Forbidden on the app / blank page	NGINX can't read the build dir: re-run the `chmod o+x /opt …` line; confirm `angular/dist/angular/browser/index.html` exists.
certbot fails to validate	DNS A record not pointing to the VPS yet (`dig +short erp.mondomaine.com`), or ports 80/443 blocked by the cloud firewall.
`DisallowedHost` / 400 from Django	`ALLOWED_HOSTS` in `.env` doesn't match your domain. Fix, then `docker compose -f docker-compose.nginx.yml up -d`.
CSRF errors in admin/forms	`CSRF_TRUSTED_ORIGINS=https://erp.mondomaine.com` in `.env`.
Media (logos) 404	The `/media/` `alias` path in the NGINX config must match the compose bind-mount `deploy/vps/media`.
Redirect loop (too many redirects)	**IP-only (Mode A):** you must set `SECURE_SSL_REDIRECT=0`, `SESSION_COOKIE_SECURE=0`, `CSRF_COOKIE_SECURE=0` in `.env` (Django forces https otherwise). **Domain:** NGINX must send `proxy_set_header X-Forwarded-Proto $scheme;` (shipped config does).
Frontend calls the wrong API	You forgot step 5 (set `apiUrl`) **before** building in step 6. For IP-only use `'/api/v1'`. Rebuild.
Build fails: `Temporary failure resolving 'deb.debian.org'`	Docker **DNS** problem on the VPS — see the fix just below.

Docker build network failures (DNS, MTU, registry timeouts)

Symptoms during `up -d --build`, all caused by the VPS's outbound network, **not** the app: - `Temporary failure resolving 'deb.debian.org'` (apt can't resolve) → **DNS**. - `failed to resolve source metadata … dial tcp …:443: i/o timeout` when pulling the base image → **TCP connection stalls**, almost always an **MTU** problem (large TLS packets dropped on a tunneled/NAT'd VPS).

Step 1 — Is it MTU? Test whether full-size (1500) packets pass:

```
ping -M do -s 1472 -c 3 1.1.1.1      # 1472 + 28 = 1500
# If that fails but this works, your path MTU is < 1500 (the cause):
ping -M do -s 1372 -c 3 1.1.1.1      # 1372 + 28 = 1400
```

Step 2 — Give Docker public DNS + a smaller MTU, then restart:

```
echo '{ "dns": ["8.8.8.8","1.1.1.1"], "mtu": 1400 }' | sudo tee /etc/docker/daemon.json
sudo systemctl restart docker
```

(If step 1 only passed at `-s 1372` or lower, use `"mtu": 1280` — the safest value.)

Step 3 — Build with the *classic* builder. The `daemon.json` MTU applies to the normal Docker bridge, but **BuildKit** (lines like "load local bake definitions") uses its own network and ignores it. Force the old builder so the MTU applies, and pre-pull the base image through the daemon first:

```
sudo docker pull public.ecr.aws/docker/library/python:3.12-slim
cd /opt/erp/deploy/vps
export DOCKER_BUILDKIT=0 COMPOSE_DOCKER_CLI_BUILD=0
sudo -E docker compose -f docker-compose.nginx.yml build web
sudo docker compose -f docker-compose.nginx.yml up -d
```

With BuildKit off, once the base image is local the build won't re-contact the registry.

Fallback — ECR unreachable: pull the same image from Docker Hub and retag it to the name the Dockerfile expects:

```
sudo docker pull python:3.12-slim
sudo docker tag python:3.12-slim public.ecr.aws/docker/library/python:3.12-slim
DOCKER_BUILDKIT=0 COMPOSE_DOCKER_CLI_BUILD=0 sudo -E docker compose -f docker-
compose.nginx.yml build web
```

As a sanity check, confirm the host itself has outbound access (outside Docker): `curl -I https://public.ecr.aws` and `curl -I http://deb.debian.org`.

Useful commands:

```
# Backend logs
docker compose -f docker-compose.nginx.yml logs -f web
# NGINX logs
sudo tail -f /var/log/nginx/error.log /var/log/nginx/access.log
# Restart everything
docker compose -f docker-compose.nginx.yml restart
```

Files used by this guide (in the repo)

- `deploy/vps/docker-compose.nginx.yml` — DB + backend (NGINX variant).
- `deploy/vps/nginx-erp.conf` — host NGINX site config for a **domain** (HTTP; certbot adds HTTPS).
- `deploy/vps/nginx-erp-ip.conf` — host NGINX site config for **IP only** (HTTP, no domain).
- `deploy/vps/nginx-erp-ip-https.conf` — **IP only** over **HTTPS** with a self-signed cert.

- `deploy/vps/.env.example` — environment template (copy to `.env`; includes the IP-only toggles).

> The repo also ships a **Caddy** variant (`deploy/vps/docker-compose.yml` + `Caddyfile`) that does HTTPS automatically in a container. This guide intentionally uses **NGINX** instead, as requested.